Intro to Cryptography with Rust

Dr. Cyprian Omukhwaya Sakwa

May 2, 2025

What is Cryptography?

- Cryptography is about **hiding and protecting information**.
- You use it every day – when using WhatsApp, visiting a secure website, or making crypto payments.
- Cryptography makes sure:
  - Only the right people can read a message (called **confidentiality**).
  - You can check if a message was changed (called **integrity**).
  - You know who sent the message (**authentication**).
- It's like sending a secret message in a locked box – and only the person with the key can open it.

Why Use Rust for Cryptography?

- Rust is a programming language designed for **safety and speed**.
- When writing cryptography, you want to avoid mistakes like:
  - Reading memory you shouldn't read.
  - Crashing because of bugs.
- Rust helps avoid these errors at **compile time**, before the program runs.
- It's fast like C++, but much safer.

Rust Makes Crypto Safer

- Rust checks for errors while you write code. This is important in cryptography where a small mistake can be dangerous.
- It helps you write code that uses multiple CPU cores safely, which is good for speeding up encryption or decryption.
- Rust doesn't use a garbage collector, so your programs run fast and predictably.

Real Cryptography in Rust

- Rust already has great tools (libraries) for doing crypto:
  - `rustls` – secure websites without needing OpenSSL.
  - `curve25519-dalek` – encryption with modern elliptic curves.
  - `tfhe-rs` – compute on encrypted data (homomorphic encryption).
  - `pqcrypto` – future-proof crypto that even resists quantum computers.
- These libraries are built with Rust's safety and performance in mind.

A Simple Example – Encrypting a Message

Encrypt a secret message in just a few lines of Rust:

```
use chacha20poly1305::ChaCha20Poly1305, aead::*;
let key = ChaCha20Poly1305::generate_key(&mut OsRng);
let cipher = ChaCha20Poly1305::new(&key);
let nonce = ChaCha20Poly1305::generate_nonce(&mut OsRng);
let ciphertext = cipher.encrypt(&nonce, b"top secret".as_ref())?;
```

This encrypts the message `"top secret"` so no one else can read it.
Note: Don't reuse the same key and nonce!

What is a Nonce?

- A **nonce** stands for "*number used once*".
- It is a unique value (usually random or a counter) used only once per encryption operation.
- Ensures that even if you encrypt the same message twice, you get different outputs.
- **Purpose:** Prevent replay attacks and ensure message uniqueness.

Warning: Reusing a nonce with the same key can break encryption security!

What This Code Does

- Uses `ChaCha20-Poly1305`, a fast and secure AEAD cipher.
- Generates a random key and nonce with `OsRng`.
- Encrypts a plaintext message in memory.
- Immediately decrypts it using the same key and nonce.
- Prints the ciphertext (in hex) and recovered plaintext.

What Does "Encrypts a Message in Memory" Mean?

"Encrypts a plaintext message in memory" means that the encryption happens entirely inside your program's RAM, not involving files, databases, or network transmission.

Breakdown:

- **Plaintext:** A message like `b"Confidential data goes here."` is stored in a variable.
- **Key and nonce:** Are also generated and stored in memory (using `OsRng`).
- **Encryption:** The cipher takes the plaintext and nonce, and returns the ciphertext—all within the running program.
- **No files involved:** The data is not read from or written to disk — it's just variables in the program.
- **Temporary:** Once the program ends, all of it (key, nonce, plaintext, ciphertext) disappears unless you explicitly save it.

Who Runs This Code?

- As written, it is run by someone who has access to:
  - The plaintext
  - The secret key
- Meant for:
  - Local encryption/decryption
  - Testing and learning the algorithm

Production Use: Split Responsibilities

In a real-world application, you would split this into two roles:

Encryptor (Sender)

- Generates a key (or derives from password)
- Encrypts plaintext using a secure nonce
- Outputs or transmits: `nonce || ciphertext`

Decryptor (Receiver)

- Has the key (or derives it)
- Receives the ciphertext and nonce
- Decrypts to recover the original plaintext